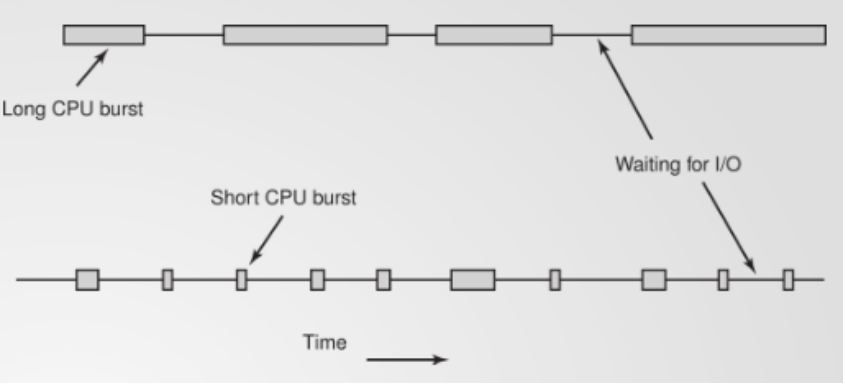
9_14_aprile

CPU BURST → indica il tempo in cui la CPU è impegnata a fare i calcoli di un processo



La parte vuota significa che sta aspettando il risultato di un operazione di I/O quindi magari la CPU va a svolgere altri compiti di altri processi

Obiettivi degli algoritmi di scheduling

Per trovare un algoritmo ottimale andremo a definire diverse metriche, queste saranno utili per definire degli algoritmi ottimali.

Definiamo diversi ambienti: batch, interattivi e real-time, questo perché in base allo scopo che devono perseguire i sistemi operativi si differenziano tra loro quindi lo farà anche lo scheduler:

1. I sistemi *Batch* sono sistemi progettati per svolgere un grande lavoro in blocco senza molta interazione con l'utente: ad esempio un super computer che elabora una simulazione, gli vengono forniti tutti i dati e il computer parte a svolgere il suo lavoro
2. I sistemi *Interattivi* sono quelli più comuni e devono interagire molto con l'utente (botta e risposta) sono i sistemi operativi dei cellulari o dei personal computer
3. I sistemi *Real-time* sono sistemi che non hanno molta interazione con l'utente e la caratteristica principale e la velocità e il rispetto delle scadenze temporali: ad esempio il sistema che gestisce tutti i controlli elettronici di un auto come l'ABS che deve rispondere in maniera tempestiva oppure il pilota automatico di un aereo

Obiettivi comuni tra i sistemi:

- equità nell'assegnazione della CPU;
- bilanciamento nell'uso delle risorse;

● Obiettivi tipici dei sistemi batch:

- massimizzare il throughput (o produttività);
- minimizzare il tempo di turnaround (o tempo di completamento);

- minimizzare il tempo di attesa;

● Obiettivi tipici dei sistemi interattivi:

- minimizzare il tempo di risposta;

● Obiettivi tipici dei sistemi real-time:

- rispetto delle scadenze;
- prevedibilità.

Generalità

Ricordiamo

Preemptive = Il sistema operativo può interrompere a metà un processo in esecuzione per dare la CPU a qualcun altro.
Non-preemptive = Una volta che un processo ottiene l'uso della CPU, se la tiene stretta finché non ha finito.

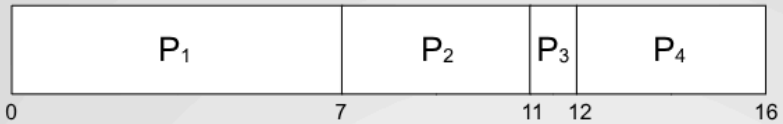
Scheduling nei sistemi batch

Abbiamo diversi algoritmi che vengono usati nei sistemi batch:

- **FCFS**(First-come first-served) - non preemptive: il primo che arriva è il primo che viene servito, praticamente una coda FIFO molto semplice ma non troppo utile
- **SJF**(Shortest job first) - non preemptive: presuppone la conoscenza del tempo impiegato da ogni lavoro (viene eseguito il job che ha durata minore) - si riduce il tempo minimo di attesa, ma resta comunque non ottimale
- **SRTN**(Shortest Remaing Time Next) - preemptive: È la versione "preemptive" del SJF. Se un processo lungo sta girando e all'improvviso arriva un processo cortissimo, il sistema ferma quello lungo per sbrigare subito quello corto.

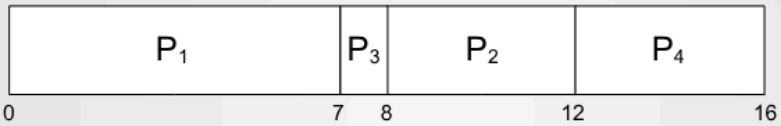
(I numeri di arrivo e durata sono "quanti di tempo")

- **FCFS**:

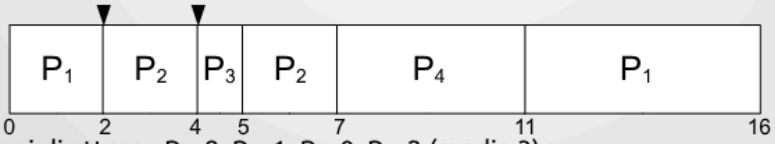


Processo	Arrivo	Durata
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

  - tempi di attesa: P_1=0; P_2=5; P_3=7; P_4=7 (media 4.75);
  - tempi di completamento: P_1=7; P_2=9; P_3=8; P_4=11 (media 8.75);
- **SJF**:



  - tempi di attesa: P_1=0; P_2=6; P_3=3; P_4=7 (media 4);
  - tempi di completamento: P_1=7; P_2=10; P_3=4; P_4=11 (media 8);
- **SRTN**:



  - tempi di attesa: P_1=9; P_2=1; P_3=0; P_4=2 (media 3);
  - tempi di completamento: P_1=16; P_2=5; P_3=1; P_4=6 (media 7).

Basandoci su questi esempi vediamo che SRTN è il più efficiente perché confrontando i tempi di arrivo e il restante tempo di completamento decide quale processo eseguire essendo preemptive può bloccare l'esecuzione di un processo per eseguirne un altro, questo unito alla caratteristica di SJF (cioè shortest job first) lo rendono il migliore

Scheduling nei sistemi interattivi

Anche qui abbiamo diversi algoritmi per ottimizzare questo problema, uno di questo è:

- **Scheduling round robin**: (RR): questa è una versione con prelazione del FCFS, che si basa sullo gestire il tutto in un quanto di tempo

Questa versione funziona in base a dei quanti di tempo o time slice che di solito hanno una durata di 20-50ms
In sostanza ogni processo può occupare la CPU per un time slice appena finisce il tempo il processo verrà messo in fondo alla coda (FCFS usa una coda FIFO) e viene eseguito il processo successivo, se il processo completa il suo lavoro prima della fine del quanto di tempo allora libererà la CPU e inizierà l'esecuzione di un altro processo

Esempio:

Immagina di avere in background un rendering video pesantissimo e in primo piano Word, dove tu stai digitando. Se non ci fosse il Round Robin, il tuo clic sulla tastiera

> rimarrebbe bloccato ad aspettare per minuti interi che il video finisca di calcolare. Con il Round Robin, il rendering video lavora per (ad esempio) 20 millisecondi, poi viene sbattuto in fondo alla fila; tocca a Word, che elabora la tua lettera in 1 millisecondo e rilascia la CPU, poi tocca di nuovo al video, ecc.

Continua nella prossima lezione...